

Lab 1 – Perfect Numbers

Requirements

Write a procedural C++ program which determines all the “Perfect” numbers less than or equal to the maximum value to be tested which is provided by the user. The user can either specify the maximum value on the command line, or when the program prompts the user for the value. If the command line argument is not a valid number, the user will also be prompted for the maximum value to test.

A perfect number is one where the sum of the factors of the number (excluding the number itself) is **equal** to the number. For example, the number 8 has factors of 1,2,4,8 and $1+2+4=7$ therefore 8 is not a perfect number. The number 28 has factors of 1,2,4,7,14,28 and $1+2+4+7+14=28$ therefore 28 is a perfect number.

Your program must implement two functions that are contained within a module (.h and .cpp file pair), the functions are highlighted in the C version of `main()` below; the parameters and return values of your C++ functions `GetMaxValue()` and `IsPerfect()` must match what is shown below. Note that your program must use a C++ version of `main()` which you must write and not the C version given below.

```
int main(int argc, char* argv[])
{
    /* The number from the command line or user prompt */
    long int MaxNumber;

    /* Get maximum number */
    MaxNumber = GetMaxValue(argc, argv);

    /* Loop through all the numbers up to the maximum */
    for (long int lcv = 1; lcv <= MaxNumber; lcv++)
    {
        /* Print out the number being tested but don't advance the line */
        printf("%ld\r", lcv);

        /* Check to see if the number is perfect or not */
        if (IsPerfect(lcv) == 1)
        {
            /* The number is perfect, print it out on a new line */
            printf("%ld\n", lcv);
        } /* if() */
    } /* for() */

    /* Print out blanks to over-write the display of the last number tested */
    printf("          \r");

    return 0;
} /* main() */
```

Assessment Criteria

- 6pts – Complete and accurate implementation of the design specifications
- 2pts – Hierarchical module files
- 2pts – Proper comments and code structure
- EXTRA CREDIT 3pts – Also include a function **IsWeird()** which determines if a number is a “Weird” Number and print the list of all weird numbers up to the maximum number entered by the user.

A weird number is one where the sum of its factors (excluding the number itself) is **greater** than the number, and there **does not exist** a subset of the factors that add up to be equal to the number. For example, the number 18 has factors of 1,2,3,6,9,18 and $1+2+3+6+9=21$ (which is greater than 18) but there is a subset of factors (3,6,9) that sum to 18, therefore 18 is not a weird number. The number 70 has factors 1,2,5,7,10,14,35,70 and $1+2+5+7+10+14+35=74$ (which is greater than 70) and there is no subset of factors which sum to 70, therefore 70 is a weird number.

Submission Requirements

- Submit only your source code (i.e., main C++ source code file, module C++ source code file and module header file) – do not submit the entire Visual Studio project
- A document containing screenshots of your program with different cases to demonstrate full functionality (PDR preferred)

Note that I will be compiling and running your program, the maximum grade is 1pt if I cannot compile and run your program from the files that you have submitted.

Objective

The purpose of this lab is to introduce C++ code and hierarchical (module-based) source code partitioning.